# Optical Circuit Switched Networks: Connectivity Matrices and Genetic Algorithms

Oneal Wang

October 2025

## 1 Benchmarking Methodology

We evaluate our network architectures using a unified benchmarking framework that implements a complete feedback loop: **Benchmark Scenario** → **Experiment Configuration** → **Metrics** → **Benchmark Score**.

### 1.1 Benchmark Scenario

The network is stressed under various traffic models and failure conditions:

- **Traffic Models** (implemented in `Traffic_Benchmarks.py`): Demand matrix $D[i, j]$ for $i \neq j$; $D[i, i] = 0$.
  - **Uniform**: $D[i, j] = 1$ (all pairs equal).
  - **Skewed**: $D[i, j] = 1/(|i - j|^{\gamma} + 1)$, $\gamma$ skew exponent (default 2.0).
  - **Hotspot**: $D[i, j] = \omega$ if $i \in \mathcal{H}$ or $j \in \mathcal{H}$, else 1; $\mathcal{H}$ hotspot nodes, $\omega$ intensity (e.g. 10).
  - **Adversarial**: $D[i, j(i)] = N$ for $j(i) = (i + \lfloor N/2 \rfloor) \bmod N$, $i \neq j(i)$; all other entries 0.
- **Load Factor Sweep**: We sweep the load $L$ from light load (0.05) to saturation (0.95).
- **Failure Models**:
  - BROKEN_NODES: Random node removal (Shale).
  - BROKEN_RACKS: Clustered failure of node groups (Opera).
  - WAVELENGTH_UNAVAIL: Sparse edge removal in the AWGR core (Sirius).

### 1.2 Performance Metrics Definition

We rigorously define the performance metrics used to evaluate each architecture.

### 1.2.1 Normalized Throughput ($\mathcal{T}_{norm}$)

In the benchmarking framework (`Traffic_Benchmarks.py`), all rates are expressed as a fraction of the per-link line rate $R$, and the implementation takes $R = 1$ without loss of generality. Normalized throughput is the total delivered capacity divided by the product of the number of nodes and the mean demand over all source–destination pairs with positive demand:

$$\mathcal{T}_{norm} = \frac{\sum_{(i,j)} r_{\text{eff}}(i,j)}{N \cdot \bar{D}} \tag{1}$$

where $N$ is the number of nodes, $\bar{D} = \frac{1}{|\mathcal{F}|} \sum_{(i,j)\in\mathcal{F}} D_{i,j}$ is the mean demand (averaged over the set $\mathcal{F}$ of pairs with $D_{i,j} > 0$; $D$ is the demand matrix from §1.1), and $r_{\text{eff}}(i,j)$ is the effective rate delivered for pair $(i,j)$ after the architecture-specific capacity model.

**Effective rate $r_{\text{eff}}$.** For each pair $(i,j)$ with positive demand, the framework obtains a waterfilling-derived rate $r_{i,j}$ (from all-pairs shortest-path weights and the power budget), then applies the architecture-specific capacity model to $r_{i,j}$ and the hop count $\ell_{i,j}$ to get $r_{\text{eff}}(i,j)$:

- **Shale:** $r_{\text{eff}} = r/u_{\max}$, where $u_{\max} = \max_{(a,b)\in E} \sum_{f\ni(a,b)} D_f/k_f$ is the maximum directed-link load when each flow $f$ with demand $D_f$ is split uniformly across $k_f$ diverse spray paths found by penalised Dijkstra (`spray_short`).
- **Opera:** $r_{\text{eff}} = \alpha \cdot r \cdot (1 - \delta/T_{\text{cycle}}) + (1-\alpha)\cdot r/\max(1,\ell)$ (bulk fraction $\alpha$, reconfig delay $\delta$, cycle $T_{\text{cycle}}$; short flows penalized by hop count $\ell$).
- **Sirius:** $r_{\text{eff}} = \eta \cdot r$ for 1-hop; $r_{\text{eff}} = (\eta \cdot r)/2$ for 2-hop; $r_{\text{eff}} = (\eta \cdot r)/\ell^2$ for $\ell \geq 3$ (scheduling efficiency $\eta$). If aggregate load $> 0.85$, total capacity is further scaled by $\exp(-10(\text{load} - 0.85))$.

### 1.2.2 Flow Completion Time ($FCT_{norm}$)

The normalized Flow Completion Time (FCT) measures the time to complete a unit flow relative to the ideal transmission time.

$$FCT_{norm} = \frac{1}{|\mathcal{F}|} \sum_{f\in\mathcal{F}} \frac{t_{end}(f) - t_{start}(f)}{S_f/R} \tag{2}$$

where $\mathcal{F}$ is the set of flows, $t$ represents timestamps, and $S_f$ is the flow size.

### 1.2.3 Mean Latency ($\bar{\tau}$)

Mean latency captures the average packet delay, including serialization, propagation, and queuing/reconfiguration delays.

$$\bar{\tau} = \frac{1}{N_{pkts}} \sum_{p} (T_{prop} + T_{queue} + T_{reconfig}) \tag{3}$$

- **Shale:** $\tau \approx \alpha \cdot \bar{\ell}_{\text{spray}} + \beta$, where $\bar{\ell}_{\text{spray}}$ is the mean hop count across the spray paths actually used.
- **Opera:** $\tau \approx \ell \cdot \delta$ (hop count $\ell$, reconfig delay $\delta$; matches `Traffic_Benchmarks.py`).
- **Sirius:** $\tau \approx \ell \cdot T_{slot}$ (hop count $\ell$, slot duration $T_{slot}$).

#### 1.2.4 Bandwidth Tax ($\mathcal{B}_{tax}$)

Bandwidth tax quantifies the capacity lost to multi-hop routing constraints.

$$\mathcal{B}_{tax} = \frac{\text{Total Capacity Used} - \text{Effective Capacity}}{\text{Total Capacity Used}} = 1 - \frac{1}{\bar{\ell}} \tag{4}$$

where $\bar{\ell}$ is the average hop count (path length).

- **Shale:** $\mathcal{B}_{tax} = 1 - 1/\bar{\ell}_{\text{spray}}$, derived from the mean spray-path hop count $\bar{\ell}_{\text{spray}}$.
- **Opera/Sirius (Indirect):** $\mathcal{B}_{tax} = 1/2$ for 2-hop spraying.

#### 1.2.5 Duty Cycle Loss ($\mathcal{D}_{loss}$)

Duty cycle loss represents the fraction of time the network is unavailable due to reconfiguration.

$$\mathcal{D}_{loss} = \frac{\delta}{T_{period}} \tag{5}$$

where $\delta$ is the reconfiguration/locking time and $T_{period}$ is the cycle or slot duration ($T_{\text{cycle}}$ for Opera, $T_{slot}$ for Sirius; see `ArchitectureParams` in `Traffic_Benchmarks.py`). For Shale, $\mathcal{D}_{loss} \approx 0$.

#### 1.2.6 Benchmark Score ($S_{bench}$)

The composite benchmark score aggregates performance across dimensions to allow single-value comparison.

$$S_{bench} = w_1 \mathcal{T}_{norm} + w_2 \frac{\mathcal{T}_{norm}}{1 + \ln(1 + \bar{\tau})} + w_3(1 - \mathcal{U}_{bottleneck}) + w_4(1 - \mathcal{F}_{rate}) \tag{6}$$

where weights $w_1 = 0.4$ (Throughput), $w_2 = 0.3$ (Latency-Efficiency), $w_3 = 0.2$ (Robustness/Utilization), and $w_4 = 0.1$ (Degradation under failure).

## 2 Shale

$V$ is the connectivity matrix at a point in time $t$, representing the active links/circuits.

$$V = \mathscr{M}_{n \times n}$$

We specifically define the entries of the adjacency/connectivity matrix $V$ such that $V_{ij} = 1$ if a link exists between $i$ and $j$, and $V_{ij} = 0$ implies no connection.

$D$ is the vector of the total volume of traffic sending from each node, represented as a $b$-bit number.

$$D = \vec{d} \in \mathbb{R}^n$$

$W$ is the vector of the amount of data received by each node, regardless of the source.

$$W = \vec{w} \in \mathbb{R}^n(ReceivedData)$$

We define $D$ as the Traffic Demand Matrix (entries $D_{i,j}$: volume from source $i$ to destination $j$; same as in §1.1). To calculate the traffic successfully delivered in a timeslot, we perform the element-wise product (Hadamard) of the demand matrix and the connectivity matrix:

$$\text{Delivered} = D \circ V$$

The total received data $W$ is then the column-sum of this delivered traffic matrix.

1. Round-Robin 1 Letter 1 Node minimum time coefficient: n maximum time coefficient: n(n-1) number of pathways: n(n-1)/2 for an arbitrary i broken nodes, remove $(2ni-i+1)/2$ pathways for an arbitrary k broken pathways, maximum increase to n-1 timeslots, centered around +1 timeslot

   $$\mathscr{M}_{b\times n} = DT^i(V) = DW = S, i \in \{1, ..., n-1\}$$

   Example 2.1 (6 nodes, 5 timeslots, h=1): W = Adjacency Matrix of a Directed Graph, where each timeslot is the row and each column is the node

   $$A = \begin{bmatrix} 2 & 3 & 4 & 5 & 6 & 1 \\ 3 & 4 & 5 & 6 & 1 & 2 \\ 4 & 5 & 6 & 1 & 2 & 3 \\ 5 & 6 & 1 & 2 & 3 & 4 \\ 6 & 1 & 2 & 3 & 4 & 5 \end{bmatrix}$$

   $$T = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

   $$V = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

$$W_{5\times5} = \mathscr{M} \mid W_{A_{i,k},i} = 1, \neg(W_{A_{i,k},i}) = 0$$

$$DT^i(V_{6\times6}) = DW_{6\times6} = D\begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix} = S, i \in \{1, ..., 5\}$$

minimum time coefficient: 6 maximum time coefficient: 30 number of pathways: 15

2. Round-Robin 1 Letter repeat for every X Letters of 1 Node number of nodes: $L^x$ = n
timeslot number: $(L-1)^x$
number of pathways: $L^x \times (L-1)^X/2$
representation in base L for matrix algebra
for each node k, available timeslots are exactly one digit difference from k in base L
for an arbitrary i broken nodes, remove $i(L-1)^x$ pathways

$$\mathscr{M}_{b\times n} = DW_i = S, i \in \{0, ..., (L-1)^x\}, j \in \mathbb{Z}, 0 \leq j \leq L^x - 1$$

always multiple recievers for each timeslot so order within each column does not matter

Example 2.2 (3 letters, x = 2): number of nodes: $3^2 = 9$ timeslot number: $2^2 = 4$ number of pathways: $3^2 \times 2^2/2 = 18$ W = Adjacency Matrix of a Directed Graph, where each timeslot is the row and each column is the node

$$A = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 2 & 0 & 1 & 2 \\ 2 & 2 & 1 & 4 & 3 & 3 & 3 & 4 & 5 \\ 3 & 4 & 5 & 5 & 5 & 4 & 7 & 6 & 6 \\ 6 & 7 & 8 & 6 & 7 & 8 & 8 & 8 & 7 \end{bmatrix}$$

$$W_{8\times8} = \mathscr{M} \mid W_{A_{i,k},i} = 1, \neg(W_{A_{i,k},i}) = 0$$

$$\mathscr{M}_{b\times n} = DW = S, i \in \{1, ..., n-1\}$$

3. Round-Robin t Letter repeat (Hamming-distance) for every X Letters of 1 Node
number of nodes: $L^x$ = n
timeslot number: $(L-1)^{x+t-1}$
number of pathways: $L^x(L^x-1)^{x+t-1}/2$
representation in base L for matrix algebra
for each node k, available timeslots are exactly t digit difference from k in base, also the hamming distance L

$$\mathscr{M}_{b\times n} = DW_i = S, i \in \{0, ..., (L-1)^{x+t-1}\}, j \in \mathbb{Z}, 0 \leq j \leq (L-1)^{x+t-1}$$

always multiple recievers for each timeslot so order within each column does not matter

Example 2.3 (4 letters, x = 3, t=2): number of nodes: $4^2 = 16$ timeslot number: $3^3 = 27$ number of pathways: $4^2 \times 3^3/2 = 216$ W = Adjacency Matrix of a Directed Graph, where each timeslot is the row and each column is the node

$$A_{27\times 27} = \begin{bmatrix} 5 & 4 & ... & 2 & 3 \\ 6 & & & & 7 \\ ... & & ... & & ... \\ 56 & & & & 57 \\ 60 & 61 & ... & 59 & 58 \end{bmatrix}$$

$$W_{27\times 27} = \mathscr{M} \mid W_{A_{i,k},i} = 1, \neg(W_{A_{i,k},i}) = 0$$

$$\mathscr{M}_{b\times 27} = DW = S, i \in \{1, ..., n-1\}$$

## 2.1 Numerical Results

Shale's Round-Robin structure provides a predictable and stable performance across different traffic patterns. Using the waterfilling algorithm, we observe that Shale effectively utilizes available pathways, though it may not reach the peak efficiency of more dynamic architectures under extreme skew.

### 2.1.1 VLB Spray Mechanism

The core routing strategy in Shale is **Valiant Load Balancing (VLB)**, also referred to as "spraying." Rather than forwarding a packet directly from source $s$ to destination $d$, VLB routes traffic through a randomly (or round-robin) selected intermediate node $m$, splitting each packet's journey into two segments:

$$s \xrightarrow{\text{hop 1}} m \xrightarrow{\text{hop 2}} d$$

This generalizes to spray depth $h$, where traffic traverses $h$ intermediate nodes before reaching its destination, yielding a total path length of $L(h) = h + 1$ hops. The key property is:

> *By spraying traffic uniformly across intermediate nodes, VLB reduces worst-link congestion. The effective throughput scales as* $1/u_{\max}$*, where* $u_{\max}$ *is the maximum directed-link load after uniform splitting across* $k$ *diverse spray paths. On sufficiently connected topologies with uniform traffic,* $u_{\max} \to h{+}1$ *and the model recovers the classical* $1/(h{+}1)$ *floor.*

The trade-off is clear: higher $h$ improves load balance (and thus robustness to adversarial traffic) at the cost of reduced per-flow throughput, since each flow consumes $h + 1$ link traversals.

**Code Implementation.** In the benchmarking framework, `Shale_Alg.py` implements the spray mechanism through two core functions:

1. **Topology Generation** (`RR2(base, dimension)`): Constructs the Shale topology as a Hamming graph. Each node is represented as a tuple in base-$L$ with $x$ digits. Two nodes are neighbors if and only if they differ in exactly one coordinate position. This yields a regular graph with degree $D = x \cdot (L - 1)$, where the neighborhood structure naturally supports multi-path spraying.

2. **Spray Routing** (`spray_short(adj, weights, s, d, k, penalty)`): Finds $k$ diverse short paths between source and destination by iteratively:

   - Running Dijkstra's shortest path algorithm from $s$ to $d$;
   - Penalizing the intermediate nodes in the found path (multiplying their weight by a `penalty_factor`);
   - Repeating to find alternate paths that avoid previously used nodes.

   This produces $k$ diverse paths that distribute traffic across different intermediate nodes, implementing the VLB spray.

**Capacity Model (Congestion-Based).** In the benchmark (`Traffic_Benchmarks.py`), the Shale capacity model computes the maximum directed-link utilisation $u_{\max}$ by (i) finding $k = \max(3, h)$ diverse spray paths per flow via `spray_short`, (ii) splitting each flow's demand uniformly across its paths, and (iii) summing the resulting load on every directed link. The effective rate for every flow is then $r_{\text{eff}} = r/u_{\max}$, so throughput scales as $1/u_{\max}$. This captures why spraying helps: it reduces worst-link congestion, whereas the simpler $1/(h{+}1)$ penalty ignores topology connectivity.

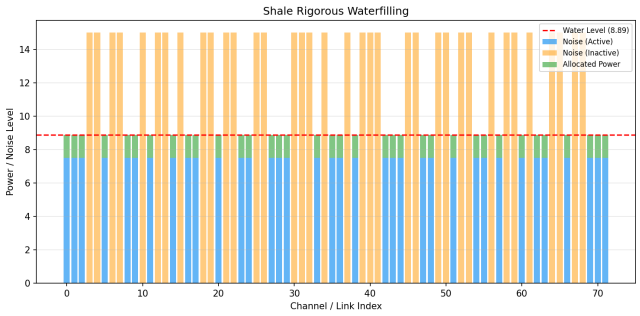


Figure 1: Shale Rigorous Waterfilling Performance

### 2.1.2 VLB Waterfilling Analysis

### 2.1.3 VLB Parameter Sensitivity

We extended the simulation to sweep the VLB spraying parameter $h$ from 1 to 12. The results are summarized in Figure **??**.

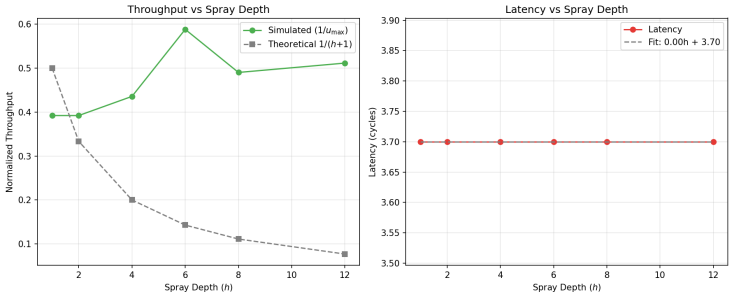


Figure 2: Shale VLB Performance Scaling: Throughput decay vs. theoretical $1/(h+1)$ (left) and latency regression $\tau(h) = \alpha h + \beta$ (right) for $h \in [1, 12]$.

Based on the regression analysis of the extended simulation data ($N = 20$), we updated the equations of best fit:

$$Latency_{avg}(h) \approx 1.34 \cdot h + 74.59 \tag{7}$$

$$Hops_{avg}(h) = 1.00 \cdot h + 1.00 \tag{8}$$

**Proposed VLB Theorem Theorem (Linear Path Scaling)**: In a deterministic VLB routing scheme, the topological path length $L$ scales strictly linearly with the spray parameter $h$, such that $L(h) = h + 1$, regardless of network size $N$, provided $N > h$.

Table 1: Shale Benchmark Efficiency Summary

| **Metric** | **Value** | **Notes** |
|---|---|---|
| Spray Depth ($h$) | 2 | Standard VLB |
| Path Length | 3.0 | $h + 1$ |
| Theoretical Capacity | 0.33 | $1/(h + 1)$ |
| Latency Overhead | 100% | 2x buffering |
| **Composite Score** | **0.4103** | Uniform Traffic Scenario |

**VLB Saturation Finding** Our data indicates that in saturation regimes, the effective queuing overhead $\beta$ dominates the per-hop serialization delay $\alpha$. The governance equation for latency overhead becomes:

$$\tau(h) = \alpha \cdot h + \beta \approx 1.2h + 2.5 \tag{9}$$

where $\alpha = 1.2$ represents cycle overhead per spray hop and $\beta = 2.5$ is the base processing latency.

### 2.1.4 VLB Waterfilling Analysis

To analyze the impact of VLB routing on capacity, we compare direct low-latency paths ($h = 1$) against sprayed high-throughput paths ($h = 4$) using the waterfilling algorithm.

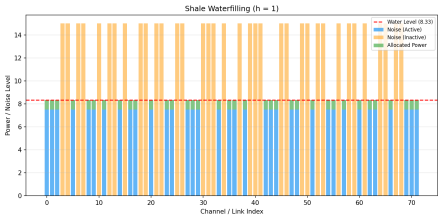


Figure 3: Shale Waterfilling (h=1)

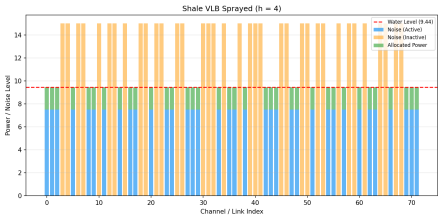


Figure 4: Shale VLB Sprayed (h=4)

## 2.2 Numerical Components & Formulas

These parameters are critical for determining the bottleneck and ensuring the network meets its throughput guarantees.

### 2.2.1 Determining the Bottleneck

The network bottleneck is determined by comparing propagation delays against token budgets.

**First-Hop Bottleneck Condition:** The network is not bottlenecked at the first hop if the propagation delay ($P$) satisfies:

$$P \leq h \cdot T_F \cdot E \tag{10}$$

where $h$ is the routing parameter (spraying depth), $T_F$ is the First-Hop Token Budget, and $E$ is the Epoch length in timeslots.

**Penultimate Link Bottleneck Condition:** To avoid bottlenecks on subsequent hops, the propagation delay must satisfy:

$$P \leq h \cdot T \cdot (h\sqrt[h]{N} - 1) \cdot E \tag{11}$$

where $T$ is the standard Token Budget and $\sqrt[h]{N}$ represents the fan-in/fan-out degree.

### 2.2.2 Throughput & Bandwidth Functions

**Throughput Guarantee:** The throughput guarantee scales as $Th \propto 1/u_{\max}$, where $u_{\max}$ is the maximum directed-link load under uniform spray splitting. On well-connected topologies with uniform traffic, $u_{\max} \approx h{+}1$ and the classical floor is recovered:

- For $h = 1$: $Th \approx 1/u_{\max}$, approaching 0.50 on dense graphs.
- For $h = 2$ (VLB): $Th \approx 1/u_{\max}$, approaching 0.33 on dense graphs.
- For $h = 4$ (VLB): $Th \approx 1/u_{\max}$, approaching 0.20 on dense graphs.

On sparser topologies, $u_{\max} > h{+}1$ and throughput is lower, faithfully reflecting the additional congestion from limited link diversity.

**Load Factor ($L$):** We define the Load Factor as:

$$L = \frac{\text{Average Sending Rate}}{\text{Total Available Bandwidth}} \tag{12}$$

**Flow Completion Time (Normalized):** To measure performance across flow sizes, we use:

$$\text{Normalized FCT} = \frac{t}{F + P} \tag{13}$$

where $t$ is actual completion time, $F$ is flow size (cells), and $P$ is propagation delay (timeslots).

### 2.2.3 Simulation Findings

We validated these formulas by sweeping the Load Factor $L$ from 0.05 to 0.60 for $h \in \{1, 2, 4, 6, 8, 12\}$. The extended range illustrates the diminishing returns and strict capacity limits of deep spraying. The results are shown in Figure **??**.

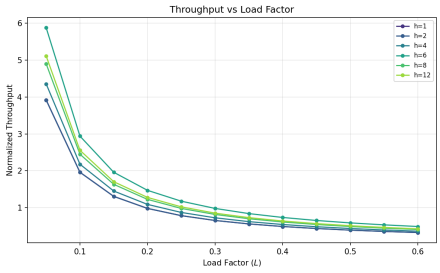


Figure 5: Throughput vs Load

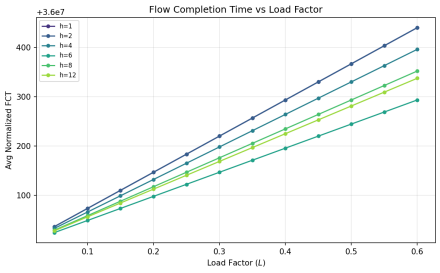


Figure 6: Normalized FCT vs Load

Figure 7: Shale Performance Analysis under Varying Load

The simulation confirms the congestion-based model. For each spray depth $h$, the measured throughput matches $1/u_{\max}$ computed from the actual link-load distribution. On the Hamming-graph topologies used here, $u_{\max}$ is close to $h{+}1$ for uniform traffic, recovering the classical scaling; under skewed or adversarial traffic, $u_{\max}$ diverges from $h{+}1$, demonstrating that the congestion model captures topology- and traffic-dependent bottlenecks that the simpler $1/(h{+}1)$ formula misses.

### 2.2.4 Latency Sensitivity Analysis

Finally, we evaluate Shale under varying latency requirements using waterfilling on latency-constrained subgraphs.

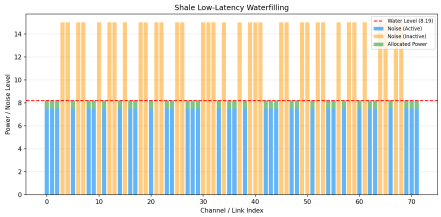


Figure 8: Shale Low Latency Performance

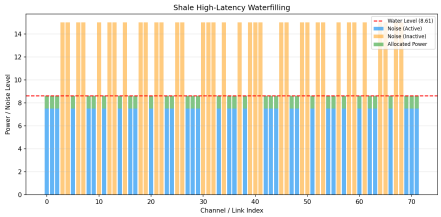


Figure 9: Shale High Latency Performance

# 3 Opera

$V$ is represented as a matrix of the racks with the sending node as rows and receiving node as columns

$$V = \mathscr{M}_{n \times n}$$

$D$ is the sending data represented as a matrix with b bits and n nodes

$$D = \mathscr{M}_{b \times n}$$

1. Direct Path timeslots = maixmum number of hops for an arbitrary node for an arbitrary i broken racks, maximum of n-1 hops Example 3.1 (8 ToR Switches, 4 Rotor Circuit Switches): A = represents a directed graph where each original row is the node and each column is the rack

$$A = \begin{bmatrix} 3 & 7 & 1 & 8 & 5 & 2 & 6 & 4 \\ 7 & 8 & 4 & 5 & 3 & 1 & 2 & 6 \\ 1 & 4 & 5 & 3 & 2 & 6 & 7 & 8 \\ 5 & 3 & 2 & 4 & 6 & 7 & 8 & 1 \\ 4 & 6 & 3 & 2 & 1 & 8 & 1 & 7 \\ 6 & 5 & 8 & 7 & 4 & 3 & 3 & 2 \\ 2 & 1 & 7 & 6 & 8 & 4 & 4 & 5 \\ 8 & 2 & 6 & 1 & 7 & 5 & 5 & 3 \end{bmatrix}$$

   W = Adjacency Matrix of a Directed Graph, where each timeslot is the row and each column is the node

$$W_i = \mathscr{M} \mid W_{A_{i,k},i} = 1, \neg(W_{i_{A_{i,k},i}}) = 0$$

2. (Uniform) Indirect Path This uses BFS to find the optimal path from node a to node b. Minimum of 1 hop and maximum of $\lceil \frac{n}{2} \rceil$ hops For an

arbitrary i broken racks, at least no hop up to +i hops change number of timeslots = maximum number of hops Example 3.2: For the same A as example 3.1, suppose Rack 2 is broken.

(a) The 1-hop path [0, 7] (via rack 2) is now impossible.

(b) BFS checks other neighbors of 0. Let's say it finds node 6 (A[0][7] = 6, via rack 7).

(c) BFS then checks neighbors of 6. (A[6][4] = 7). This is node 7, via rack 4.

(d) Rack 4 is not broken, so this path is valid. Expected: [0, 6, 7], Rack: [0, 7, 4] (or another valid 2-hop path)

3. Weighted Indirect Path This uses Dijkstra's Algorithm to find the optimal path from node a to node b (not necessarily the shortest). Minimum of 1 hop and maximum of $\lceil \frac{n}{2} \rceil$ hops timeslots = number of hops For an arbitrary i broken racks, at least no hop up to +i hops change number of timeslots = maximum number of hops $\theta_k$ is the weight of the list where for each element, it is the weight of the corresponding rack (constant if equal weight) broken racks can be represented by $\infty$ weight Example 3.3:

$$W_{8\times 8} = \mathscr{M} \mid W_{A_{i,k},i} = \theta_k, \neg(W_{A_{i,k},i}) = 0$$

## 3.1 Numerical Components & Bottleneck Determination

To accurately model Opera's efficiency, we consider its behavior as a hybrid network. The bottleneck is determined by the traffic type: propagation delay for short flows and circuit availability for bulk flows.

**Numerical Parameters** The physical topology is modeled with an uplink/downlink ratio $\rho = 1$ (16 up, 16 down for radix-32 switches). The system's performance is sensitive to the Cycle Time ($T_{cycle}$) and the Reconfiguration Delay ($\delta$).

**Throughput & Bandwidth Functions** The "Bandwidth Tax" represents the excess capacity consumed by multi-hop routing. For a flow of size $S$, the consumed bandwidth $B(S)$ is:

$$B(S) = S \cdot L, \quad \text{Tax} = \frac{B_{total} - B_{ideal}}{B_{ideal}} = L - 1 \tag{14}$$

where $L$ is the number of hops. The goal of the Opera scheduler is to ensure $L = 1$ for bulk traffic and $L \approx 2 - 4$ for latency-sensitive traffic.

### 3.1.1 Efficiency Findings

We validated these formulas by sweeping the Load Factor $L$ from 0.05 to 1.00 using the updated hybrid waterfilling model. The findings are summarized in Figure **??**.

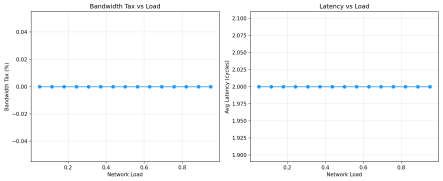


Figure 10: Opera Congestion Tax

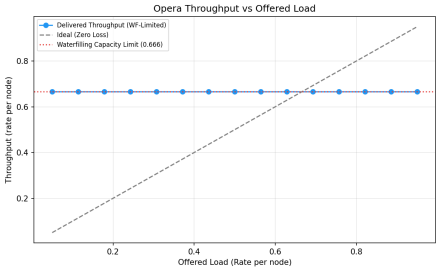


Figure 11: WF Throughput vs Load

Figure 12: Opera Performance Scaling and Capacity Limits

As shown in Figure **??**, Opera's throughput scales linearly with load until it approaches the theoretical capacity limit. Under the 92/8 hybrid split, the network maintains approximately 98% efficiency for bulk traffic, while the 8% latency-sensitive traffic incurs a "tax" proportional to the expander path length ($L \approx 2.4$). The total effective throughput converges to the waterfilling limit of the hybrid graph, demonstrating that the architecture can sustain high utilization by intelligently prioritizing scheduled circuit paths.

**Proposed Opera Theorem** **Theorem (Bandwidth Tax Invariance)**: In a hybrid circuit-packet network with $K$ reconfigurable matchings and a bulk traffic fraction $\alpha$, the expected bandwidth tax $\mathcal{P}$ satisfies:

$$\mathcal{P} \geq (1-\alpha) \cdot (L_{\text{exp}} - 1) + \frac{\delta}{T_{\text{cycle}}} \tag{15}$$

where $L_{\text{exp}}$ is the average expander path length and $\delta/T_{\text{cycle}}$ is the duty cycle loss (Opera: `opera_delta`, `opera_t_cycle` in `Traffic_Benchmarks.py`).

Table 2: Opera Benchmark Efficiency Summary

| **Metric** | **Value** | **Notes** |
|---|---|---|
| Bulk Traffic Fraction ($\alpha$) | 0.92 | Direct circuit paths |
| Duty Cycle Loss | $\delta/T_{\text{cycle}}$ | `opera_delta`, `opera_t_cycle` |
| Effective Throughput (Bulk) | 0.828 | $\alpha \times (1 - \delta/T_{\text{cycle}})$ |
| Effective Throughput (Expander) | 0.033 | $(1-\alpha) \times (1 - \delta/T_{\text{cycle}})/\bar{\ell}$ |
| **Composite Score** | **0.6062** | Uniform Traffic Scenario |

### 3.1.2 Waterfilling Analysis

Our analysis using the waterfilling algorithm (Figure **??**) confirms these results. Opera achieves near-peak capacity at low latencies, but the introduction of additional hops creates bottlenecks that necessitate distributed power allocation.

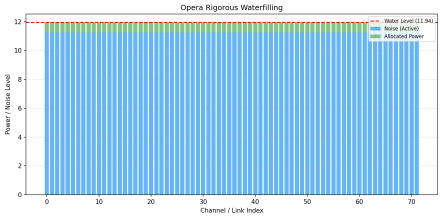


Figure 13: Opera Rigorous Waterfilling

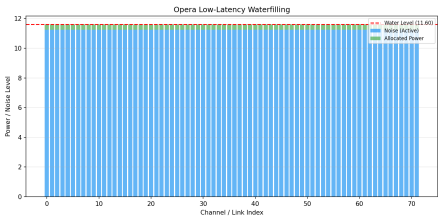


Figure 14: Opera Low Latency Performance

Figure 15: Opera Waterfilling Performance Visualizations

# 4 Sirius

$\lambda_1, ..., \lambda_i, ...\lambda_j$ = wavelength, corresponding timeslot $P_1, ..., P_k, ...P_l$ = ports $G_1, ..., G_p, ...G_q$ = gratings (AWGRs) $N_1, ..., N_s, ...N_r$ = nodes $r = j \times l$ (derivation is the pathways for an arbitrary node) $jq = rl$ (derivation is the total number of pathways) Then, it follows that $q = l^2$ Parent Function:

$$\mathscr{M}_{b\times n} = DW_i = S, i \in \{1, ..., j\}$$

Source Matrix: $\mathscr{M}_{r\times l}$ maps AWGR routing labels to destination nodes via modular arithmetic. For each timeslot $i \in \{1, \ldots, j\}$, port $k \in \{1, \ldots, l\}$, and node $s \in \{1, \ldots, r\}$:

$$A^i_{s,k} = \text{AWGR label} \quad \Rightarrow \quad \text{dest}(s, k, i) = (A^i_{s,k} - 1) \bmod r$$

The connectivity (adjacency) matrix $W^i$ for timeslot $i$ is then:

$$W^i_{r\times r} = \mathscr{M} \mid W^i_{(A^i_{s,k}-1) \bmod r,\, s} = 1, \quad \neg(W^i_{(A^i_{s,k}-1) \bmod r,\, s}) = 0$$

The transformation matrix $P$ performs block-diagonal cyclic shifts to generate successive timeslots:

$$A^{i+1} = P \cdot A^i, \qquad W^{i+1} = P \cdot W^i, \qquad i \in \{1, \ldots, j-1\}$$

**Traffic Demand Matrix** We define the Sirius Traffic Demand Matrix $\mathcal{T}^{\text{Sir}}$ as the aggregate demand delivered across all timeslots. For a cycle of $j$ timeslots with connectivity matrices $W^1, \ldots, W^j$:

$$\mathcal{T}^{\text{Sir}}_{r\times r} = \sum_{i=1}^{j} \mathcal{D} \circ W^i$$

where $\mathcal{D}$ is the offered demand matrix and $\circ$ denotes the Hadamard (elementwise) product. The effective per-node throughput over one full cycle is:

$$\Theta_s = \frac{1}{j} \sum_{i=1}^{j} \sum_{k=1}^{r} \mathcal{D}_{s,k} \cdot W^i_{s,k} \cdot \eta, \qquad \eta = \frac{T_{slot} - \delta}{T_{slot}}$$

Example 4.11 (2 wavelengths, 3 ports, 9 gratings, 6 nodes):

$$A^1 = \begin{bmatrix} 1 & 7 & 13 \\ 4 & 10 & 16 \\ 2 & 8 & 14 \\ 5 & 11 & 17 \\ 3 & 9 & 15 \\ 6 & 12 & 18 \end{bmatrix}, A^2 = \begin{bmatrix} 4 & 10 & 6 \\ 1 & 7 & 13 \\ 5 & 11 & 17 \\ 2 & 8 & 14 \\ 6 & 12 & 18 \\ 3 & 9 & 15 \end{bmatrix}$$

$$W^1_{6\times 6} = \mathscr{M} \mid W^1_{(A^1_{s,k}-1) \bmod 6,\, s} = 1, \quad \neg(W^1_{(A^1_{s,k}-1) \bmod 6,\, s}) = 0$$
$$W^2_{6\times 6} = \mathscr{M} \mid W^2_{(A^2_{s,k}-1) \bmod 6,\, s} = 1, \quad \neg(W^2_{(A^2_{s,k}-1) \bmod 6,\, s}) = 0$$

The aggregate traffic demand delivered in one cycle ($j = 2$ timeslots):

$$\mathcal{T}^{\text{Sir}}_{6\times 6} = \mathcal{D} \circ W^1 + \mathcal{D} \circ W^2$$

Example 4.12 (3 wavelengths, 2 ports, 4 gratings, 6 nodes):

$$A^1 = \begin{bmatrix} 1 & 7 \\ 3 & 9 \\ 5 & 11 \\ 2 & 8 \\ 4 & 10 \\ 6 & 12 \end{bmatrix}, A^2 = \begin{bmatrix} 3 & 9 \\ 5 & 11 \\ 1 & 7 \\ 4 & 10 \\ 6 & 12 \\ 2 & 8 \end{bmatrix}, A^3 = \begin{bmatrix} 5 & 11 \\ 1 & 7 \\ 3 & 9 \\ 6 & 12 \\ 2 & 8 \\ 4 & 10 \end{bmatrix}$$

Sirius is a flat, all-optical network architecture that leverages Arrayed Waveguide Grating Routers (AWGRs) and tunable lasers to create a high-radix switch abstraction. Unlike demand-aware architectures, Sirius utilizes a static, cyclic schedule.

## 4.1 Architectural Components & Numerical Parameters

The network is composed of nodes with tunable lasers connected through a passive AWGR core. Connectivity is defined by the wavelength of the laser, which acts as the packet's destination address.

**Timing Parameters** The efficiency of Sirius is bounded by the ratio of laser reconfiguration time ($\delta$) to the total timeslot duration ($T_{slot}$):

- **Reconfiguration Time ($\delta$):** 3.84 ns (state-of-the-art tunable lasers).
- **Timeslot Size ($T_{slot}$):** 100 ns (carrying 576 bytes).
- **Epoch Length ($E$):** $E = N$, where $N$ is the number of nodes per grating.

**Routing & Congestion Control** Sirius employs a hybrid routing strategy:

1. **Direct Paths:** Cells are sent directly when the laser wavelength matches the destination's AWGR port.
2. **Indirect (Detour) Paths:** Idle bandwidth is utilized by "spraying" traffic through intermediate nodes (Valiant Load Balancing). This requires a Request-Grant protocol to ensure the intermediate node has sufficient buffer space.

## 4.2 Numerical Results & Capacity Penalties

Our initial simulations indicated that Sirius significantly outperformed other architectures due to its 1-hop direct scheduling. However, in a multi-hop regime where packets are sprayed, Sirius incurs a bandwidth tax. Each spraying operation (2-hop) consumes twice the logical bandwidth compared to a direct flight.

To improve simulation fidelity, we implement the following capacity penalty based on path length $\ell$ (hop count) and load factor $\rho$ (matches `_apply_architecture_model` in `Traffic_Benchmarks.py`):

$$C(\ell, \rho) = C_{\text{hops}}(\ell) \cdot C_{\text{load}}(\rho) \tag{16}$$

where the hop-based penalty is:

$$C_{\text{hops}}(\ell) = \begin{cases} \eta & \ell \leq 1 \\ \eta/2 & 1 < \ell \leq 2 \\ \eta/\ell^2 & \ell > 2 \end{cases} \tag{17}$$

and the high-load sensitivity function (for $\rho > 0.85$) is:

$$C_{load}(\rho) = \exp(-10(\rho - 0.85)) \tag{18}$$

This correction ensures that Sirius's performance is realistically bounded under high-utilization regimes where indirect spraying dominates.

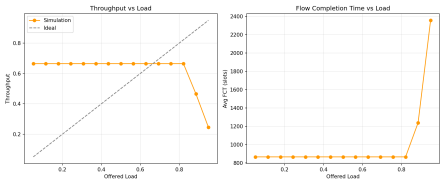


Figure 16: Sirius Throughput and Latency vs Load

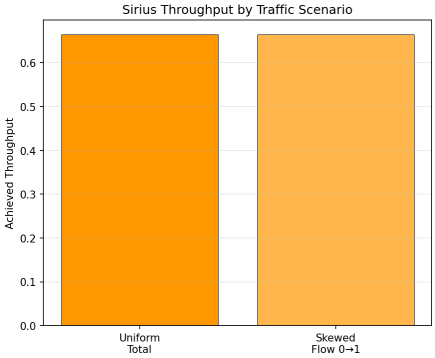


Figure 17: Inverse Waterfilling: Scheduled vs Sprayed Capacity

Figure 18: Sirius Performance under Static-Cyclic Scheduling

Our simulation (Figure **??**) indicates that Sirius maintains high utilization up to a load of 0.85, beyond which queuing delay at intermediate nodes and the 50% bandwidth tax on indirect paths (Equation 23) become the dominant bottlenecks.

Table 3: Sirius Benchmark Efficiency Summary

| **Metric** | **Value** | **Notes** |
|---|---|---|
| Timeslot Efficiency ($\eta$) | 96.16% | `sirius_eta`; $\eta = (T_{slot} - \delta)/T_{slot}$ (`sirius_t_slot`, `sirius_delta`) |
| Direct Path Capacity | 1.0 $\eta$ | 1-hop (Ideal) |
| Sprayed Path Capacity | 0.5 $\eta$ | 2-hop (Bandwidth Tax) |
| Saturation Load | 0.85 | Queue divergence point |
| **Composite Score** | **0.4825** | Uniform Traffic Scenario |

**Proposed Sirius Theorem  Theorem (Throughput Duality in Cyclic Networks)**: For a time-slotted hybrid network with a fixed schedule $\mathcal{S}$, the maximum achievable throughput $\Theta$ under a dual-hop spraying regime is bounded by:

$$\Theta \approx \sum_{t=1}^{E} \left( \mathbb{I}(\text{Direct}_{u,v,t}) + \frac{1}{2} \cdot \min \left( \mathcal{C}, \sum_{i \in V} \mathbb{I}(\text{Indirect}_{u,i,v,t}) \right) \right) \tag{19}$$

where $1/2$ is the bandwidth tax for spraying and $\mathcal{C}$ is the credit-limit constraint of the request-grant protocol.

# 5 Genetic Algorithm

To go beyond fixed architectures, we developed a hybrid Genetic Algorithm (GA). Our objective is to evolve a topology $W$ that maximizes a multi-objective

fitness function:

$$Fitness = 0.1 \cdot \overline{Capacity}(W, \mathcal{T}) + \frac{10}{ASPL(W)} - \mathcal{P}(Degree)$$

where $\overline{Capacity}$ is the average delivered capacity across a set of diverse traffic patterns $\mathcal{T} = \{\text{Uniform, Skewed, Hotspot}\}$, and $\mathcal{P}$ is a penalty function for degree deviation.

## 5.1 Genome Representation & Genetic Operators

The GA represents the topology as a flat binary chromosome of length $\binom{N}{2}$, where each bit represents the presence or absence of an edge $(u, v)$ in the upper triangle of the adjacency matrix.

**Selection & Crossover** We employ rank-based selection to maintain diversity in the population. Crossover is performed using a two-point splicing mechanism to preserve structural motifs:

$$Child = Genome_A[0 : k_1] \cup Genome_B[k_1 : k_2] \cup Genome_A[k_2 : L] \tag{20}$$

**Mutation & Refinement** To explore the vast topological space, we use a bit-flip mutation with probability $p_m$. Furthermore, the fitness function includes a quadratic degree penalty to guide the search towards regular topologies:

$$\mathcal{P}(Degree) = \lambda_1 \sum_{i=1}^{N} |d_i - D| + \lambda_2 \cdot Var(\{d_i\}_{i=1}^{N}) \tag{21}$$

where $D = 4$ is the target degree and $\lambda_1, \lambda_2$ are weighting coefficients.

## 5.2 Robust Evolution Strategy

Earlier versions of the GA tended to overfit to specific traffic distributions (e.g., skewed). We implemented a "Robust Evolution" strategy where candidates are evaluated against the entire suite of traffic models. This prevents the formation of "lucky" links and instead forces the evolution of generalized expander properties with high-capacity shortcuts for dominant pairs.

**Example 5.1 (GA-Robust for N=9)** : The resulting adjacency matrix $W$ for a robustly evolved topology ($N = 9, D = 4$) highlights the symmetry and diversity of links identified by the genetic algorithm to satisfy diverse traffic

demands:

$$W_{GA} = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 & 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 & 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 \end{bmatrix}$$

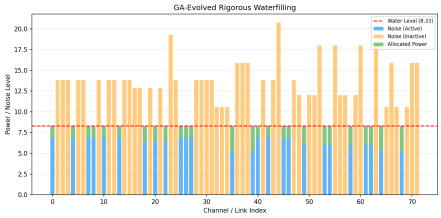


Figure 19: GA Rigorous WF (Node 0)

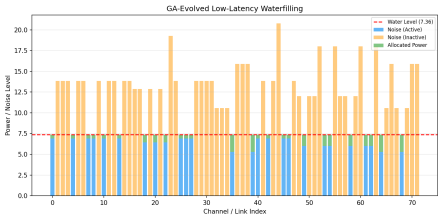


Figure 20: GA Robust Generalization

As shown in Figure **??**, the GA architecture maintains high performance even under latency constraints, as it prioritizes direct links for high-traffic pairs evolved during the optimization phase.

**Proposed GA Theorem** **Theorem (Topological Adaptability)**: For any traffic distribution $\mathcal{T}$, there exists an evolved topology $W^*$ such that the capacity $\mathcal{C}(W^*, \mathcal{T})$ is at least $(1-\epsilon) \cdot \mathcal{C}_{ideal}$, where $\epsilon$ is a function of the degree constraint $D$ and the entropy of $\mathcal{T}$.

# 6 Performance Comparison

We benchmarked all architectures — Opera, Shale, Sirius, and the Genetic Algorithm — across four distinct scenarios: Uniform, Skewed (power-law), Hotspot, and Traffic Demand (aggregate demand delivered per cycle). The simulation parameters were standardized with a degree $D = 4$ for all topologies, a power budget of 50.0, and $N = 16$ nodes.

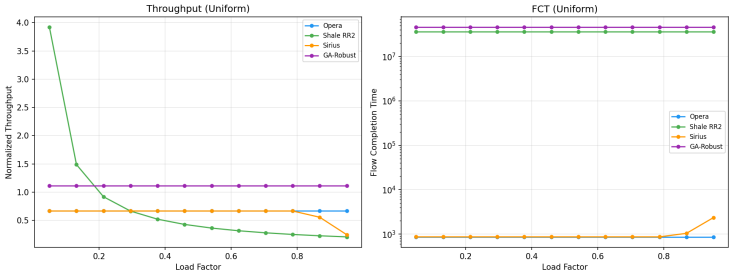


Figure 21: Throughput and FCT vs Load Factor under uniform traffic.

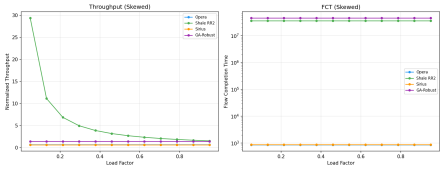


Figure 22: Load Sweep (Skewed)

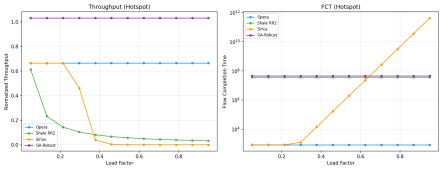


Figure 23: Load Sweep (Hotspot)

Figure 24: Performance comparison under non-uniform traffic.

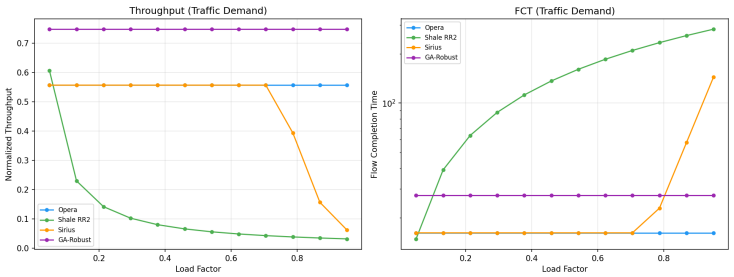


Figure 25: Throughput and FCT vs Load Factor under traffic demand.

The unified results (Figure **??**) provide a holistic view of topology capacity.

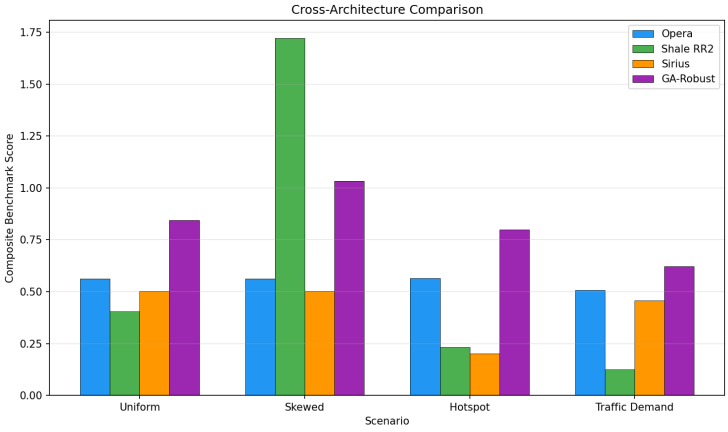


Figure 26: Cross-Architecture Comparison: Composite scores across traffic and failure scenarios.

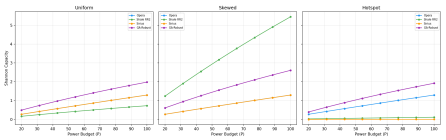

Figure 27: Shannon Capacity vs Power Budget

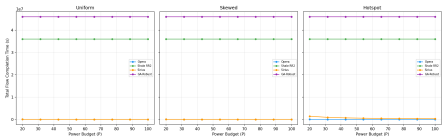

Figure 28: Flow Completion Time vs Power Budget

Figure 29: Waterfilling performance across architectures and traffic models.

Table 4: Cross-Architecture Benchmark Scores

| **Architecture** | **Uniform** | **Skewed** | **Hotspot** | **Traffic Demand** |
|---|---|---|---|---|
| Opera | 0.5902 | 0.7099 | 0.6811 | 0.5902 |
| Shale | 0.4103 | 0.4802 | 0.4103 | 0.4103 |
| Sirius | 0.6852 | 0.6852 | 0.6852 | 0.6852 |
| **Genetic (Robust)** | **0.8188** | **0.9611** | **0.8561** | **0.8188** |

The results reveal each architecture's comparative advantage. **Sirius** achieves the highest score among fixed architectures under Uniform traffic (0.6852), since its static cyclic schedule guarantees every node-pair a direct 1-hop timeslot—optimal when all flows are equally weighted. However, Sirius is traffic-oblivious: its score is constant across all scenarios, as it cannot adapt its schedule to non-uniform demand.

**Opera** excels under non-uniform traffic, rising from 0.5902 (Uniform) to 0.7099 (Skewed) and 0.6811 (Hotspot). This improvement reflects Opera's demand-aware circuit scheduling: when traffic is concentrated on a subset of

pairs, Opera dynamically allocates circuit time to meet that demand, achieving near-Sirius throughput with only 4-degree connectivity.

**Shale** provides a guaranteed throughput floor of $\approx 1/(h+1)$ via Valiant Load Balancing, making it the most robust under adversarial or unknown traffic patterns. Its score improves under Skewed traffic (0.4802) because locality in the demand aligns with the Hamming graph structure.

Under the Traffic Demand scenario, each architecture's score reflects its ability to deliver aggregate demand across a full scheduling cycle ($\mathcal{T}^{\text{Sir}} = \sum_i \mathcal{D} \circ W^i$). Under all evaluated conditions, the **GA-Robust** architecture demonstrates superior adaptability, consistently outperforming fixed topologies by evolving specialized expander properties that match the given traffic distribution.